

湖北汽车工业学院

HUBEI UNIVERSITY OF AUTOMOTIVE TECHNOLOGY

智能网联汽车学院

移动终端软件开发

实验指导书

制定人：张　毅

审核人：张吴波

日　期：2026年8月

目 录

目 录.....1
实验二　数据存储技术.....2
一、实验内容.....2
二、思考题.....5

实验二　数据存储技术

一、实验内容

程序 1：SharedPreferences 数据存取

（1）写出主 Activity 保存数据、第二个 Activity 读取数据的关键代码。

```
// 主 Activity：把 EditText 中的数据保存到 SharedPreferences，再跳转
Button login = (Button) findViewById(R.id.login);
login.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String username = usernameET.getText().toString();
        String password = passwordET.getText().toString();
        SharedPreferences sp = getSharedPreferences("mrsfot", MODE_PRIVATE);
        SharedPreferences.Editor editor = sp.edit();
        editor.putString("username", username);
        editor.putString("password", password);
        editor.commit();
        Intent intent = new Intent(SpMainActivity.this, SpReadActivity.class);
        startActivity(intent);
    }
});
// 第二个 Activity：从同一个 SharedPreferences 中读取并显示
SharedPreferences sp = getSharedPreferences("mrsfot", MODE_PRIVATE);
String username = sp.getString("username", "mr");
String password = sp.getString("password", "001");
usernameTV.setText("用户名：" + username);
passwordTV.setText("密　　码：" + password);
```

（2）粘贴输入数据和读取显示结果的运行截图。



图 1-1　主 Activity 输入用户名 wwh、密码 123456



图 1-2　第二个 Activity 读取并显示 SharedPreferences 中的数据

程序 2：SQLite 数据库操作

（1）写出创建数据表以及增、删、查、改的核心代码。

```
// 1. 创建数据表 person（DBHelper.onCreate）
db.execSQL("CREATE TABLE IF NOT EXISTS person (_id INTEGER PRIMARY KEY "
       + "AUTOINCREMENT, name VARCHAR, age INTEGER, info TEXT)");
// 2. 增：事务中循环插入 4 条记录（DBManager.add）
db.beginTransaction();
for (Person person : persons)
    db.execSQL("INSERT INTO person VALUES(null, ?, ?, ?)",
            new Object[]{person.name, person.age, person.info});
db.setTransactionSuccessful();
db.endTransaction();
// 3. 删：删除年龄大于等于 30 的记录（DBManager.deleteOldPerson）
db.delete("person", "age >= ?", new String[]{String.valueOf(person.age)});
// 4. 改：按姓名把年龄改为 80（DBManager.updateAge）
ContentValues cv = new ContentValues(); cv.put("age", person.age);
db.update("person", cv, "name = ?", new String[]{person.name});
// 5. 查：查询全部记录并封装成 List（DBManager.query）
Cursor c = db.rawQuery("SELECT * FROM person", null);
while (c.moveToNext()) {
    Person person = new Person();
    person.name = c.getString(c.getColumnIndex("name"));
    person.age = c.getInt(c.getColumnIndex("age"));
    person.info = c.getString(c.getColumnIndex("info"));
    persons.add(person);
}
c.close();
```

（2）粘贴增删改查的运行结果截图，简要标注操作。



图 2-1　add：新增 A、B、C、D 四条记录



图 2-2　update：把 D 的年龄改为 80



图 2-3　delete：删除年龄≥30 的 C、D



图 2-4　query：查询显示剩余 A、B

程序 3　ContentProvider 数据共享

（1）写出 Provider 的 insert、query 方法及客户端通过 ContentResolver 调用它们的核心代码。

```
// 已在 AndroidManifest.xml 中注册，authorities 与客户端 Uri 一致
// MyContentProvider.insert：插入数据并返回 Uri
SQLiteDatabase db = dbHelper.getWritableDatabase();
long rowId = db.insert("User", null, contentValues);
if (rowId > 0) {
    Uri rowUri = ContentUris.withAppendedId(uri, rowId);
    getContext().getContentResolver().notifyChange(rowUri, null);
    return rowUri;
}
throw new SQLException("Failed to insert row into " + uri);
// MyContentProvider.query：查询全部数据
SQLiteDatabase db = dbHelper.getReadableDatabase();
Cursor c = db.rawQuery("SELECT * FROM User", null);
c.setNotificationUri(getContext().getContentResolver(), uri);
return c;
// 客户端：通过 ContentResolver 插入数据
ContentValues values = new ContentValues();
values.put("USER_NAME", "wwh");
getContentResolver().insert(CONTENT_URI, values);
// 客户端：通过 ContentResolver 查询并显示结果
Cursor cur = getContentResolver().query(CONTENT_URI,
       new String[]{"_id", "USER_NAME"}, null, null, null);
while (cur.moveToNext()) {
    String id = cur.getString(cur.getColumnIndex("_id"));
    String name = cur.getString(cur.getColumnIndex("USER_NAME"));
    resultTV.append("_id=" + id + "  USER_NAME=" + name + "\n");
}
```

（2）粘贴插入数据和查询显示结果的运行截图。



图 3-1　插入 wwh、hujunfeng 后查询显示的结果（_id=1、_id=2）

二、思考题

2.1 习题

1. 写入数据到 SharePreferences 的步骤和读取 SharePreferences 中的数据步骤分别是什么？

写入：

1. 在 SpMainActivity 中调用 getSharedPreferences("mrsfot", MODE_PRIVATE) 获取 SharedPreferences 实例。

2. 调用该实例的 edit() 方法得到 Editor，用 putString("username", 用户名) 和 putString("password", 密码) 写入键值。

3. 调用 commit() 提交，返回 true 表示写入成功，随后用 startActivity 跳转到 SpReadActivity。

读取：

1. 在 SpReadActivity 中用同一个文件名 "mrsfot" 和 MODE_PRIVATE 获取 SharedPreferences 实例。

2. 调用 getString("username", "mr")、getString("password", "001") 按键读取，键不存在时使用默认值。

3. 把读取到的用户名和密码显示到界面。

SharedPreferences 以 XML 键值对形式保存在应用私有目录，适合保存少量配置；本实验用 commit() 同步提交，也可用 apply() 异步提交。

2. 添加一条记录到 SQLite 数据表有哪 2 种方法？

我用到两种方法：

1. insert() 方法：先取得 SQLiteDatabase，把 name、age、info 放进 ContentValues，再调用 db.insert("person", null, values)。成功时返回新记录的 rowId，失败返回 -1，适合插入单条记录。

2. execSQL() 方法：调用 db.execSQL("INSERT INTO person(name, age, info) VALUES(?, ?, ?)", new Object[]{name, age, info})，直接执行带占位符的 INSERT 语句，不返回 rowId，但占位符可防止 SQL 注入。

两种方式本质都是执行 INSERT，只是封装层次不同，插入前都要确保 person 表已创建。本实验的批量 add 方法就是在事务中循环 execSQL() 插入四条记录的，插入后我还会用 query() 核对结果。

3. SQLiteHelper 的作用是什么？怎么使用它？

我理解题目中的 SQLiteHelper 就是 SQLiteOpenHelper。它的作用是管理数据库的

创建与版本升级：首次调用 getWritableDatabase() 时回调 onCreate() 建库建表；版本号增大时回调 onUpgrade() 升级表结构；它还负责数据库的打开、缓存和关闭。

使用时：

1. 定义 DBHelper 继承 SQLiteOpenHelper，构造方法调用 super(context, "test.db", null, 1)，指定库名和版本。

2. 重写 onCreate()，执行 CREATE TABLE person(_id INTEGER PRIMARY KEY AUTOINCREMENT, name VARCHAR, age INTEGER, info TEXT)；重写 onUpgrade()，先 DROP TABLE 再重建。

3. 在 DBManager 中实例化 DBHelper，通过 getWritableDatabase()（只读时用 getReadableDatabase()）取得 SQLiteDatabase 后增删改查，用完由 DBManager 的 closeDB() 关闭数据库。这样数据库按需打开，版本变化时也不易出错。

4. ContentProvider 的作用是什么？如何使用它？

作用：ContentProvider 是 Android 四大组件之一，用于跨应用共享数据。它用 content:// 开头的 Uri 标识数据，把增删改查封装成统一接口；客户端通过 ContentResolver 按 Uri 找到对应 Provider 并调用其方法，无需了解底层存储，同时便于做权限控制。

用法：

1. 编写 MyContentProvider 继承 ContentProvider，重写 onCreate()、getType()、insert()、update()、delete()、query()；内部用 CpDbHelper 操作 Users.db 的 User 表，insert 时由 db.insert 得到 rowId，再用 ContentUris.withAppendedId 生成 rowUri 并 notifyChange。

2. 在 AndroidManifest.xml 的 application 内注册 provider，authorities 设为 com.example.datastorage.mycontentprovider。

3. 客户端 CpMainActivity 定义 CONTENT_URI = Uri.parse("content://com.example.datastorage.mycontentprovider")，用 getContentResolver().insert() 插入 wwh、hujunfeng，再用 query() 查询并将 _id、USER_NAME 显示出来。authorities 与 Uri 必须一致，否则会找不到 Provider。

2.2 实验小结

1 编程遇到的问题及解决方案（编程中遇到的问题以及程序调试过程等，分条写）

1. 现象：SpReadActivity 总显示默认值 mr/001。原因：读取时文件名或键名与写入时不一致。解决：把 "mrsfot"、username、password 提取为常量，写入后检查 commit() 返回值，再核对两个 Activity 的取值。

2. 现象：点击 add 后 Logcat 报 no such table: person。原因：建表 SQL 的字段类型或主键自增写法有误，且改表后未提升版本号。解决：按 person(_id INTEGER PRIMARY KEY AUTOINCREMENT, name VARCHAR, age INTEGER, info TEXT) 修正 onCreate，把版本号加 1 或卸载重装，让建表重新执行。

3. 现象：查询有数据但 ListView 空白。原因：SimpleAdapter 的 from 键与 Map 的键不一致，或 to 里的控件 id 与 simple_list_item_2 不匹配。解决：把 Map 的键统一为 "name"、"info"，from 数组写成 {"name","info"}，to 数组写成 {android.R.id.text1, android.R.id.text2}，并在每次操作后重新设置适配器刷新列表。

4. 现象：客户端报 IllegalArgumentException，提示找不到 provider。原因：清单未注册或 authorities 与 content:// 开头的 Uri 不一致。解决：在 application 中补全 provider 的 name 和 authorities，确保客户端 CONTENT_URI 完全一致，再用 Logcat 定位。

2 实验收获及体会（本次实验掌握了哪些知识点，学到了什么？可以进行总结在这里，分条写，不要堆在一起）

1. 掌握了 SharedPreferences 的键值存储：用 getSharedPreferences() 加 MODE_PRIVATE 建文件，Editor 的 putString()/commit() 写，getString() 带默认值读，适合保存少量配置，也学会用默认值避免空显示。

2. 理解了 SQLiteOpenHelper 的建库建表与版本管理，学会把 DBManager 作为业务层封装 add/update/delete/query，Activity 只负责按钮事件和刷新列表，结构更清晰。

3. 掌握了 SQLite 事务和 Cursor 的使用：批量插入用 beginTransaction()/setTransactionSuccessful()/endTransaction() 保证一致性，查询后遍历 Cursor 并调用 close() 关闭，避免资源泄漏。

4. 弄清了 ContentProvider 与 ContentResolver 的配合：用 authorities 和 content:// 开头的 Uri 暴露数据，客户端通过 resolver 跨应用增查，authorities 必须与 Uri 对应，并用 Logcat 调试。

3 不足之处及下一步需改进的地方（实验不足之处，以及需要改进的地方，分条写，不要堆在一起）

1. 界面只放了 EditText、Button 和 ListView，未做空值、长度和密码强度校验，密码还以明文保存；下一步应加输入校验、提示和加密存储。

2. person 表未建索引，query() 每次全表扫描且没有分页，数据量大时会卡顿；下一步给 name、age 建索引，并用 LIMIT/OFFSET 分页、支持模糊查询。

3. Provider 只实现了基本 CRUD，未设置 read/write 权限，也没有 ContentObserver 监听数据变化；下一步补上权限声明和变化通知。

4. 功能停留在基础增删改查，界面用 SimpleAdapter 较简单；下一步可换 RecyclerView，并迁移到 Room，增加搜索、排序和单元测试。